

DOCUMENTACIÓN TÉCNICA EMOTIONAL CHATBOT

Sistema de acompañamiento emocional – Ingeniería de software II



Miguel Angel Zamudio Villa - mizamudiovunal.edu.co

Esteban Santacruz - mizamudiovunal.edu.co

Camilo Andrés Medina Sánchez - cmedinasa@unal.edu.co

Cristian Camilo Castillo Meneses - crcastillo@unal.edu.co

Daniel Santiago Rodriguez Rodriguez - drodriguezrod@unal.edu.co

1. Resumen ejecutivo

Emotional Chatbot es una solución de software orientada al análisis emocional de mensajes escritos por usuarios. El sistema busca ofrecer una respuesta inicial de acompañamiento mediante la combinación de una interfaz web moderna, un backend encargado de la lógica de negocio y un servicio de inteligencia artificial especializado en clasificación emocional. La idea central del proyecto consiste en permitir que un usuario se registre o inicie sesión, escriba un mensaje sobre su estado emocional y reciba una orientación textual apoyada por análisis automático de emociones.

A nivel técnico, la solución utiliza una arquitectura distribuida en tres componentes. El frontend está implementado con React y Astro, con una capa visual construida sobre componentes reutilizables y un estado centralizado. El backend está desarrollado con Spring Boot y Spring Security, y se encarga de la autenticación, autorización, persistencia y orquestación del flujo de conversación. El microservicio de inteligencia artificial expone rutas FastAPI y utiliza un modelo de Transformers para traducir el texto y clasificar emociones predominantes.

La documentación que se presenta a continuación describe el funcionamiento general del sistema, su arquitectura, los módulos principales, el flujo de autenticación, el mecanismo de análisis emocional, la persistencia de datos, la integración entre capas y las consideraciones de despliegue. La intención es que este documento sirva como referencia técnica para mantenimiento, evolución y comprensión del proyecto por parte de otros desarrolladores o evaluadores académicos.

2. Alcance y objetivo del documento

El alcance de esta documentación es estrictamente técnico. Se describen los componentes implementados en el código fuente, las responsabilidades de cada módulo, la manera en que se comunican entre sí y el papel que cumplen dentro de la solución global. No se trata de un manual clínico ni de una guía psicológica, sino de una explicación funcional de un sistema de software orientado al acompañamiento emocional.

El objetivo principal es ofrecer una visión ordenada y comprensible de cómo opera la plataforma. Para ello se explica el propósito de las capas de presentación, negocio y análisis, así como la relación entre autenticación JWT, consultas a base de datos, almacenamiento de conversaciones y consumo de servicios externos.

3. Descripción general de la solución

La plataforma fue concebida como un espacio de escucha inicial para personas que desean expresar cómo se sienten. El sistema recibe un mensaje en lenguaje natural, detecta la emoción dominante, calcula una distribución aproximada de emociones relacionadas y devuelve una respuesta orientativa que no pretende reemplazar la atención profesional. El valor principal del proyecto está en combinar experiencia de usuario, persistencia del historial y procesamiento automatizado.

El flujo general inicia en el frontend, donde el usuario puede autenticarse o registrar una cuenta. Una vez autenticado, el usuario escribe un mensaje en el espacio de conversación y el sistema lo envía al backend. El backend valida la identidad del usuario, llama al servicio de IA, organiza la respuesta y almacena el resultado. Posteriormente, el frontend muestra la emoción dominante, la lista de emociones detectadas y la recomendación asociada al contexto del mensaje.

4. Arquitectura general del sistema

La arquitectura del proyecto puede interpretarse como un esquema de microservicios con un backend centralizador. El frontend actúa como la capa de interacción con el usuario; el backend Spring Boot expone la API principal, administra la seguridad y registra la información; el microservicio FastAPI realiza el análisis emocional especializado. Esta separación permite que cada componente evolucione de manera relativamente independiente.

El frontend se apoya en componentes de presentación como AuthPanel, ConversationWorkspace, EmotionInsights, ConversationHistory, StatusBanner y TopBar. Cada uno resuelve una porción concreta de la interfaz y mantiene responsabilidades bien delimitadas. El componente AppScreen funciona como pantalla principal y decide qué paneles mostrar según el estado de autenticación del usuario, lo que simplifica el flujo visual de la aplicación.

En el backend, la arquitectura se organiza alrededor de controladores, servicios, repositorios, entidades y objetos de transferencia de datos. Esta forma de estructurar el código facilita la lectura, prueba y mantenimiento del sistema. Además, la integración con el servicio de IA se realiza mediante un cliente HTTP independiente, lo que reduce el acoplamiento y permite modificar el motor de análisis sin reescribir la lógica principal.

5. Tecnologías utilizadas

El proyecto combina varias tecnologías modernas que cumplen papeles complementarios. En la capa de interfaz se utilizan React 19, Astro y Tailwind CSS, lo que permite crear una experiencia visual dinámica, modular y estéticamente consistente. El uso de componentes reutilizables favorece la mantenibilidad del frontend y simplifica la composición de pantallas complejas.

En el backend se emplean Java 21, Spring Boot 3.5, Spring Security y validaciones Jakarta. Esta combinación proporciona una base robusta para exponer servicios REST, administrar la autenticación y persistir información relacional. El uso de BCrypt como codificador de contraseñas y JWT como mecanismo de acceso refleja una estrategia de seguridad estándar y ampliamente aceptada en aplicaciones web modernas.

Para la inteligencia artificial se utiliza FastAPI junto con la biblioteca Transformers y GoogleTranslator. El modelo j-hartmann/emotion-english-distilroberta-base permite clasificar emociones a partir de texto, mientras que la traducción automática ayuda a normalizar entradas en español hacia inglés antes de ejecutar la inferencia. En persistencia se observa el uso de PostgreSQL para usuarios y conversaciones, y la presencia de MongoDB en el repositorio del proyecto para estructuras más flexibles asociadas al historial.

6. Requerimientos funcionales

El sistema debe permitir el registro de usuarios con nombre completo, correo electrónico y contraseña. También debe permitir el inicio de sesión mediante correo y contraseña, devolviendo un token de autenticación para acceder a recursos protegidos. Estas capacidades se encuentran soportadas por los DTOs de autenticación y por la lógica implementada en AuthService.

El sistema debe aceptar mensajes de texto enviados por usuarios autenticados y generar una respuesta emocional basada en análisis automático. El módulo de conversación recibe el texto, invoca la IA, guarda el resultado y retorna un objeto estructurado con la información de la interacción. Además, debe ofrecer acceso al historial de conversaciones del usuario, ordenado cronológicamente.

Otro requerimiento importante es la visualización del estado emocional. La interfaz presenta una emoción dominante junto con porcentajes para cada emoción detectada. Este comportamiento se observa en los componentes de conversación e insights, donde el sistema transforma la información técnica de la respuesta en una representación visual comprensible para el usuario final.

7. Requerimientos no funcionales

Desde el punto de vista no funcional, el sistema debe ser usable, estable y fácil de mantener. La interfaz debe ser comprensible para usuarios sin conocimientos técnicos avanzados, mientras que el backend debe preservar una estructura modular que facilite cambios futuros. La separación entre frontend, backend e IA contribuye a ese objetivo.

La seguridad también es un requerimiento esencial. El proyecto contempla autenticación con JWT, sesiones sin estado, cifrado de contraseñas y protección de rutas privadas. El manejo de CORS está configurado para entornos de desarrollo local, permitiendo la comunicación entre puertos comunes de frontend como 3000, 4321 y 5173, sin comprometer la seguridad del despliegue final.

8. Diseño de la autenticación

El módulo de autenticación está construido sobre dos operaciones principales: registro e inicio de sesión. El DTO `RegisterRequest` valida nombre, correo y contraseña, mientras que `LoginRequest` exige credenciales válidas para iniciar sesión. La respuesta de autenticación se encapsula en `AuthResponse`, que devuelve un token tipo Bearer. Estos elementos conforman un contrato claro entre frontend y backend.

`AuthService` centraliza la lógica de autenticación. Durante el registro, el correo se normaliza, se verifica que no exista previamente y se almacena el usuario con la contraseña cifrada. Durante el inicio de sesión, el correo se normaliza nuevamente, se busca el usuario en la base de datos y se compara la contraseña ingresada con la almacenada. Si todo es correcto, se genera un JWT asociado al identificador y al correo del usuario.

Este diseño es apropiado para aplicaciones web modernas porque desacopla la identidad del usuario del estado de sesión tradicional. El token se convierte en la evidencia de autenticación y puede ser reutilizado por el frontend para acceder a rutas protegidas sin necesidad de mantener una sesión de servidor persistente.

9. Gestión de usuarios

La entidad `User` representa la información principal de cada cuenta registrada. Contiene atributos esenciales como `id`, `nombre`, `correo`, `contraseña` y `fecha de creación`. El uso de restricciones como `@NotBlank`, `@Email` y `@Size` garantiza que los datos almacenados cumplan con reglas

mínimas de integridad. Además, la unicidad del correo evita duplicidades y mejora la confiabilidad del registro.

El repositorio `UserRepository` extiende `JpaRepository` y expone consultas derivadas como ``existsByCorreo`` y ``findByCorreo``. Esta abstracción permite consultar usuarios sin escribir SQL manual y simplifica considerablemente la capa de persistencia. También facilita la evolución futura del modelo de datos, ya que Spring Data genera automáticamente la implementación de las consultas definidas por convención.

Desde la perspectiva del negocio, el usuario es la entidad raíz de todo el sistema. Tanto las conversaciones como el historial dependen del usuario autenticado, y el backend usa su identificador para asociar cada análisis a la persona correspondiente. De esta forma, la plataforma conserva trazabilidad y puede recuperar conversaciones anteriores de manera personalizada.

10. Gestión de conversaciones

El módulo de conversaciones constituye el núcleo funcional del proyecto. `ConversationController` expone los endpoints ``/conversations/analyze`` y ``/conversations/history``. El primero recibe el mensaje del usuario autenticado y ordena su análisis; el segundo retorna el historial ordenado de conversaciones anteriores. Ambos métodos verifican que el usuario esté autenticado antes de delegar en la capa de servicio.

`ConversationService` implementa la mayor parte de la lógica de negocio. Primero valida la existencia del usuario mediante su identificador; luego intenta llamar al cliente IA. Si la comunicación falla, activa el análisis local de respaldo. Después transforma la información obtenida en DTOs de respuesta, serializa los detalles emocionales y guarda la conversación. Finalmente, reconstruye la respuesta que consume el frontend.

El diseño del servicio es importante porque separa la captura del mensaje, la inferencia emocional y la persistencia. En vez de mezclar estas responsabilidades dentro del controlador, la solución mantiene una capa de orquestación clara que facilita pruebas, mantenimiento y futuras extensiones. Esa decisión también permite incorporar nuevos tipos de análisis sin alterar el contrato visual del sistema.

11. Persistencia de datos

La persistencia del proyecto combina el modelo relacional con estructuras más flexibles. PostgreSQL para usuarios y conversaciones, mientras que en el repositorio general del proyecto también aparece MongoDB para otros elementos del historial. Este enfoque híbrido ofrece ventajas cuando se necesitan tanto relaciones estructuradas como almacenamiento semiestructurado.

En el modelo relacional, la entidad de conversación conserva el mensaje del usuario, la respuesta generada, la emoción dominante, la fecha de creación y la referencia al usuario propietario. Además, el análisis emocional completo se serializa en formato JSON para guardar listas y mapas sin perder detalle. Esa estrategia evita diseñar tablas excesivamente fragmentadas para un resultado que cambia según el mensaje analizado.

El uso de repositorios separados simplifica la lectura del dominio. UserRepository se encarga de usuarios y ConversationRepository maneja el historial cronológico. En caso de utilizar colecciones MongoDB, los repositorios asociados permiten recuperar mensajes por conversación o por usuario, lo que sugiere una estructura orientada a documentos para registros más dinámicos y menos rígidos.

12. Configuración de seguridad y CORS

La configuración de seguridad se apoya en Spring Security y define una aplicación sin estado. Se deshabilita CSRF, se usa SessionCreationPolicy.STATELESS y se bloquean formularios y autenticación básica. Solo el conjunto de rutas relacionadas con autenticación queda libre; el resto requiere usuario autenticado. Esto concuerda con el uso de JWT como credencial de acceso.

La configuración CORS habilita varios orígenes locales comunes de desarrollo. Esto es útil porque el frontend puede ejecutarse en entornos distintos según la herramienta utilizada, como puertos asociados a Astro, React o Vite. También se permiten credenciales, se exponen encabezados de autorización y se aceptan métodos HTTP habituales como GET, POST, PUT, PATCH, DELETE y OPTIONS.

La presencia de un bean RestTemplate en HttpClientConfig evidencia la intención de realizar llamadas salientes hacia el microservicio de IA de una manera simple y estandarizada. Esta decisión, combinada con la configuración de seguridad, establece una base sólida para la integración entre componentes separados y para el consumo de APIs internas o externas.

13. Análisis emocional con IA

El servicio de inteligencia artificial se implementa en FastAPI y expone rutas específicas para analizar mensajes. La ruta `/ia/analyze` recibe un texto y retorna la distribución de emociones; la ruta `/chat/message` agrega una capa de respuesta más completa, incluyendo recomendaciones y advertencias. Esta separación permite reutilizar la inferencia básica cuando sea necesario y ofrecer una experiencia más rica en el nivel conversacional.

La clase `IAService` utiliza el pipeline de Transformers con el modelo `j-hartmann/emotion-english-distilroberta-base`. Antes de enviar el texto al clasificador, se traduce automáticamente al inglés usando GoogleTranslator. Esta fase es relevante porque el modelo de emociones está entrenado en inglés y la traducción mejora la compatibilidad con mensajes escritos en español u otros idiomas.

El resultado del análisis se procesa en forma de mapa de emociones y puntuaciones. A partir de ese mapa se determina la emoción dominante y se devuelve una respuesta JSON con el texto original, el texto traducido, la emoción principal y los porcentajes asociados. Desde el punto de vista funcional, esto convierte una tarea de clasificación técnica en una salida fácilmente consumible por el backend y por la interfaz.

14. Interfaz de usuario

El frontend está construido a partir de paneles y componentes especializados. `AuthPanel` maneja login y registro; `ConversationWorkspace` presenta el área de escritura y la respuesta del sistema; `EmotionInsights` visualiza porcentajes y emoción dominante; `ConversationHistory` permite navegar por interacciones anteriores; `StatusBanner` informa errores o mensajes operativos; y `TopBar` muestra el estado de sesión.

`AppScreen` funciona como pantalla compuesta y decide qué bloques mostrar según el estado autenticado. Cuando el usuario no ha iniciado sesión, se prioriza el acceso y el panel de conversación. Cuando sí existe una sesión activa, se habilita el historial y la visualización detallada de insights emocionales. Este comportamiento mejora la usabilidad porque adapta la interfaz al contexto del usuario.

La estructura visual busca ser clara y de bajo esfuerzo cognitivo. Las tarjetas contienen títulos, subtítulos, estados de carga y mensajes descriptivos. Además, el texto en la interfaz insiste en que no se trata de una herramienta clínica, lo cual es coherente con el objetivo ético del proyecto y evita interpretaciones incorrectas por parte del usuario final.

15. Flujo de uso del sistema

El flujo típico comienza con la autenticación. El usuario registra una cuenta o inicia sesión, recibe un token y lo usa para acceder a las secciones protegidas. Después escribe un mensaje describiendo cómo se siente. El sistema toma ese mensaje, lo envía al backend y espera una respuesta estructurada con análisis y recomendación.

Una vez que el backend completa el procesamiento, la interfaz actualiza el panel de conversación y muestra la emoción dominante. Si existen porcentajes asociados a distintas emociones, se representan visualmente mediante barras o indicadores. Al mismo tiempo, el historial conserva el mensaje para consultas posteriores, lo que permite retomar conversaciones o revisar estados emocionales anteriores.

El flujo fue diseñado para ser simple desde la perspectiva del usuario, pero modular desde la perspectiva técnica. Cada paso está dividido entre componentes especializados, lo que permite ampliar o modificar la solución sin rehacer todo el sistema. Esa separación también facilita pruebas unitarias y pruebas de integración en cada capa.

16. Consideraciones de calidad y mantenimiento

El proyecto exhibe varias características positivas en materia de calidad. La modularidad del backend, el uso de DTOs y la separación del frontend por componentes favorecen la legibilidad. La normalización de correos y la validación de formularios reducen errores de entrada. La existencia de un mecanismo de fallback incrementa la disponibilidad percibida.

Para mantenimiento futuro, resulta importante conservar contratos claros entre frontend, backend e IA. Si se modifica la estructura JSON de alguna respuesta, las demás capas deben adaptarse de forma sincronizada. También conviene documentar adecuadamente los endpoints y mantener versiones estables de las dependencias del modelo de inteligencia artificial para evitar cambios inesperados.

Desde el punto de vista de evolución, la arquitectura ya permite ampliar el sistema con funciones como clasificación de riesgo emocional, recomendaciones más personalizadas, métricas de uso o paneles administrativos. Dado que el componente de IA está desacoplado, incluso sería posible sustituir el modelo actual por otro sin reescribir toda la aplicación.

17. Instalación y ejecución

El backend se ejecuta mediante Maven y Spring Boot. En un entorno estándar, la aplicación puede levantarse con un comando como ``mvn spring-boot:run``, siempre que se cuente con la configuración apropiada de base de datos y variables de entorno. El frontend se inicia con el gestor de paquetes correspondiente y el comando de desarrollo de Astro o React según el módulo usado.

El servicio de IA requiere instalar dependencias Python y ejecutar un servidor FastAPI con Uvicorn. Debido al uso del modelo de Transformers, la primera ejecución puede tardar más de lo normal, ya que el sistema puede descargar pesos y recursos asociados. Este comportamiento es esperable y forma parte del proceso de puesta en marcha del servicio de inferencia.

En una implementación real, la coordinación entre los tres componentes debe contemplar puertos, rutas y credenciales consistentes. También es aconsejable mantener una documentación de variables de entorno, URLs base y modos de despliegue para facilitar la reproducción del entorno por parte de otros desarrolladores.

18. Conclusiones

El sistema Emotional Chatbot representa una solución técnicamente interesante porque combina autenticación, persistencia, visualización y análisis emocional automatizado. La integración de un backend Spring Boot con un microservicio FastAPI demuestra una separación clara de responsabilidades y un enfoque moderno de arquitectura distribuida.

La solución también destaca por su intención de robustez. No se limita a depender de un modelo remoto, sino que incorpora una alternativa local para mantener la funcionalidad mínima. Esta decisión aumenta la confiabilidad general del sistema y evita que un fallo externo interrumpa por completo el servicio al usuario.

En síntesis, el proyecto ofrece una base sólida para un sistema de acompañamiento emocional inicial. Aunque no sustituye la atención profesional, sí cumple un papel de interacción útil, organizada y técnicamente bien estructurada para registrar, analizar y visualizar el estado emocional expresado por el usuario.

19. Trabajo futuro

Una evolución natural del sistema sería incorporar un historial más rico con filtros por fecha, intensidad emocional y tipo de conversación. También podría añadirse una clasificación más fina de

emociones, así como métricas acumuladas que permitan observar tendencias en el tiempo. Esas mejoras enriquecerían la experiencia del usuario y ampliarían el valor analítico de la plataforma.

Otra línea de trabajo consistiría en fortalecer la inteligencia artificial con modelos multilingües o con análisis contextual más profundo. El sistema también podría incorporar recomendaciones adaptativas según el historial del usuario, alertas de riesgo o mecanismos de derivación hacia recursos de ayuda humana cuando se detecten patrones críticos.

Finalmente, sería recomendable ampliar la capa de observabilidad con logs estructurados, monitoreo y pruebas automatizadas. Estas prácticas facilitarían la evolución del sistema, la detección temprana de errores y la validación continua de las integraciones entre frontend, backend e inteligencia artificial.

20. Anexo técnico: inventario de componentes

A continuación, se presenta un inventario textual de los componentes identificados en el código compartido. El propósito de este anexo es dejar constancia de las piezas principales de la solución y del papel que cumple cada una dentro del flujo general.

Componente	Función
AuthService	Gestiona el registro y el inicio de sesión, valida credenciales y genera tokens JWT.
SecurityConfig	Define una política stateless, deshabilita formularios y limita acceso a rutas protegidas.
CorsConfig	Permite comunicación entre frontend y backend desde orígenes de desarrollo comunes.
HttpClientConfig	Expone un RestTemplate para consumir servicios HTTP externos.
ConversationController	Publica los endpoints de análisis e historial de conversaciones.
ConversationService	Coordina la integración entre IA, persistencia y respuesta al frontend.
EmotionAnalysisService	Realiza análisis local de respaldo mediante palabras clave.
IAService	Ejecuta la inferencia del modelo de emociones en el servicio FastAPI.
AuthPanel	Interfaz para autenticación y registro.
ConversationWorkspace	Área principal para redactar mensajes y

	revisar respuestas.
EmotionInsights	Visualiza la emoción dominante y la distribución porcentual.
ConversationHistory	Muestra conversaciones previas del usuario.
TopBar	Presenta el estado de sesión y acción de cierre.
StatusBanner	Muestra errores o mensajes informativos.

21. Anexo técnico: endpoints

Endpoint	Descripción
POST /auth/register	Registra un nuevo usuario.
POST /auth/login	Autentica credenciales y devuelve token.
POST /conversations/analyze	Analiza el mensaje autenticado y guarda el resultado.
GET /conversations/history	Retorna el historial de conversaciones.
POST /chat/message	Procesa un mensaje en el servicio FastAPI.
POST /ia/analyze	Devuelve emociones detectadas por el modelo.

24. Análisis detallado de la capa de autenticación

La capa de autenticación resume de forma clara uno de los principios más importantes del proyecto: proteger la identidad de cada usuario y evitar accesos no autorizados a los recursos privados. El archivo AuthService concentra el comportamiento de registro e inicio de sesión, mientras que los DTOs LoginRequest, RegisterRequest, AuthResponse y MessageResponse definen la estructura de entrada y salida entre el frontend y el backend. Este diseño evita acoplar la interfaz al modelo interno de persistencia.

Durante el registro, el correo se normaliza antes de persistirse. Esta decisión técnica no es menor, porque reduce errores derivados de mayúsculas, espacios o diferencias de escritura. También se valida que el correo no exista previamente, lo que obliga a mantener una unicidad fuerte a nivel de negocio. La contraseña, por su parte, se almacena cifrada, de modo que la base de datos no conserva secretos en texto plano. Todo ello refuerza la seguridad básica del sistema.

En el inicio de sesión, AuthService recupera al usuario por correo, comprueba la contraseña y delega la creación del token en JwtService. Aunque el código compartido no incluyó la implementación completa de ese servicio, sí queda claro que su función es generar la credencial JWT que luego utilizará el frontend. Este patrón es típico en aplicaciones modernas porque permite mantener sesiones sin estado y escalar el backend con mayor facilidad.

La estructura de autenticación también refleja una intención pedagógica. El flujo es sencillo de entender, pero suficientemente robusto para un proyecto académico. El usuario completa un formulario, el backend valida la información, la base de datos se actualiza o consulta, y la respuesta regresa con un contrato compacto. Esta simplicidad ayuda a explicar el sistema en una sustentación técnica sin sacrificar coherencia arquitectónica.

25. Análisis detallado de la capa de seguridad

La clase `SecurityConfig` define el perímetro de protección de la aplicación. Allí se desactiva CSRF, se configura la política de sesión como `stateless` y se deshabilitan mecanismos como formulario web y autenticación básica. Estas decisiones indican que la aplicación no depende de la sesión clásica de servidor, sino de un token enviado por el cliente en cada petición protegida.

El acceso libre se limita a las rutas de autenticación, mientras que el resto de los endpoints requieren autenticación previa. Este enfoque es adecuado para APIs REST consumidas por un frontend desacoplado, ya que el navegador actúa únicamente como cliente que porta el token y no como gestor de sesión. De esta forma, Spring Security protege el backend sin imponer complejidad excesiva al frontend.

`CorsConfig` complementa la seguridad en el plano de integración entre orígenes. En desarrollo local es habitual que la interfaz y el backend corran en puertos diferentes, por eso se habilitan orígenes como 3000, 4321 y 5173. Se permiten cabeceras `Authorization` y `Content-Type`, además de métodos HTTP básicos. El resultado es una configuración práctica para entornos de desarrollo y pruebas, sin abrir innecesariamente la superficie del servidor.

`HttpClientConfig`, por su parte, expone un `RestTemplate` que sirve como cliente HTTP hacia servicios externos. En este proyecto esa capacidad es fundamental para la comunicación con el microservicio de inteligencia artificial. Aunque hoy muchas aplicaciones usan clientes reactivos o librerías más nuevas, `RestTemplate` sigue siendo suficiente cuando el objetivo es realizar llamadas simples, claras y fáciles de mantener en un sistema académico.

26. Análisis detallado de la gestión de conversaciones

ConversationController marca el inicio del flujo de análisis emocional. Sus dos endpoints responden a necesidades concretas: analizar un mensaje nuevo y consultar conversaciones anteriores. El uso de `@AuthenticationPrincipal` indica que el usuario autenticado se obtiene directamente desde el contexto de seguridad, lo que simplifica el controlador y evita que la lógica de identidad se mezcle con la lógica de negocio.

ConversationService actúa como orquestador. Primero recupera el usuario, luego intenta obtener la respuesta del motor de IA y, si no puede hacerlo, activa el análisis local. Después construye un conjunto de DTOs y persiste la conversación. Esta secuencia revela que el servicio no se limita a transportar datos: toma decisiones, maneja excepciones y transforma la información en una forma útil para la interfaz de usuario.

Los DTOs de conversación cumplen funciones distintas pero complementarias. `AnalyzeConversationRequest` transporta el mensaje del usuario y valida su longitud. `ConversationResponse` encapsula la salida completa del análisis, incluyendo identificador, mensaje, emociones, respuesta generada, advertencia y fecha. `EmotionAnalysisResponse` y `EmotionScoreResponse`, finalmente, separan la emoción dominante de las puntuaciones individuales para una visualización más ordenada.

El diseño de este módulo favorece una experiencia de usuario transparente. El frontend no necesita conocer la complejidad de la inferencia ni de la persistencia; simplemente recibe un objeto ya organizado. Eso permite que la pantalla de conversación muestre una respuesta clara y que la pantalla de historial recupere interacciones pasadas sin volver a ejecutar el análisis.

27. Análisis detallado del motor de emociones local

EmotionAnalysisService es una pieza clave de resiliencia. El servicio no depende de redes neuronales ni de llamadas externas para ofrecer una respuesta mínima. Su lógica se apoya en una normalización del texto y en la comparación contra conjuntos de palabras clave asociadas a cada emoción. Aunque es más simple que un clasificador profundo, resulta muy útil como respaldo operativo.

El método de normalización elimina tildes, convierte a minúsculas y separa tokens para compararlos de forma homogénea. Esto mejora la detección en español, donde pequeñas variaciones ortográficas podrían afectar resultados. Las categorías que utiliza el motor son alegría, tristeza, ansiedad, enojo, miedo, calma y estrés. El sistema asigna porcentajes relativos y define una emoción dominante, lo que facilita la presentación visual posterior.

La recomendación generada por el motor local no pretende ser clínica. Su texto mantiene una redacción prudente y orientada al autocuidado, por ejemplo respiración pausada, apoyo cercano, pausa breve o reflexión momentánea. Esa redacción tiene un valor importante porque alinea el producto con su propósito: acompañar, no diagnosticar. El disclaimer incluido refuerza esa frontera conceptual.

Desde la perspectiva de ingeniería, este módulo también mejora la depuración del sistema. Cuando el servicio IA falla, el análisis local permite seguir probando el flujo completo sin detener el desarrollo. Esa propiedad resulta muy útil durante pruebas, demostraciones y mantenimiento, porque evita que un componente remoto comprometa toda la cadena funcional.

28. Persistencia y estructura de datos

La capa de persistencia organiza la información en torno al usuario y sus conversaciones. La entidad User conserva los atributos necesarios para identificar la cuenta, autenticarla y auditar su creación. El uso de `@CreationTimestamp` aporta una marca temporal automática, lo cual es útil para seguimiento y control. En paralelo, los repositorios simplifican la consulta de usuarios por correo y el control de duplicidades.

ConversationRepository permite recuperar conversaciones del usuario ordenadas por fecha de creación. Esta decisión es relevante porque el historial de análisis emocional pierde sentido si no se presenta en secuencia temporal. Recuperar las interacciones de manera cronológica facilita la lectura del proceso emocional del usuario y mejora la navegación dentro de la aplicación.

El sistema también muestra un enfoque pragmático frente a la estructura de datos emocionales. En vez de fragmentar excesivamente la base de datos para almacenar cada emoción por separado, el backend serializa listas y mapas a JSON. Esta estrategia reduce complejidad de modelado y conserva la información necesaria para reconstruir el análisis posteriormente. Es una solución frecuente cuando la estructura interna del resultado puede variar entre mensajes.

Si el proyecto se amplía, la persistencia podría evolucionar hacia un esquema más sofisticado con tablas de análisis, detalle de emociones, etiquetas de seguimiento o métricas agregadas. No obstante, la versión actual ya es suficiente para sostener la funcionalidad del chatbot y conservar trazabilidad de cada interacción.

29. Arquitectura del frontend y pantalla principal

AppScreen es el componente de composición principal en la interfaz. Su responsabilidad es coordinar qué paneles se muestran según el estado del usuario y delegar acciones al store centralizado. Al montar la pantalla, se ejecuta una inicialización que recupera datos de sesión o estado persistente. Esto permite que la experiencia del usuario se mantenga coherente aun cuando se recargue la página.

Cuando el usuario no está autenticado, la pantalla muestra un panel de acceso y el espacio de conversación. Cuando la sesión existe, se habilita además el historial y el panel de insights emocionales. Esta bifurcación visual crea una interfaz adaptativa que evita sobrecargar al usuario invitado con funciones que todavía no puede usar.

La organización basada en componentes separados permite que cada pieza sea más fácil de comprender y extender. AuthPanel maneja formularios; ConversationWorkspace gestiona el mensaje y la respuesta; ConversationHistory lista interacciones previas; EmotionInsights presenta indicadores emocionales; TopBar refleja el estado de sesión; StatusBanner informa problemas o acciones completadas. En conjunto, estas piezas forman una experiencia modular y legible.

Aunque el frontend no fue entregado completo en todos sus archivos, los componentes compartidos bastan para identificar una arquitectura orientada al dominio y a la reutilización. El uso de hooks y store centralizado sugiere un enfoque pensado para mantener el estado fuera de los componentes de presentación, lo cual favorece el orden del proyecto.

30. Componentes visuales del frontend

AuthPanel implementa el formulario de registro y acceso. Su principal virtud es la simplicidad: alterna entre modos de login y registro sin abandonar el contexto visual. Además, limpia parte del estado del formulario después de registrar una cuenta, lo que mejora la experiencia del usuario. Su diseño transmite una interfaz clara y con pocas distracciones.

ConversationWorkspace concentra la interacción principal. Allí el usuario escribe el mensaje, observa la respuesta del sistema y recibe una advertencia explícita sobre el carácter no clínico de la plataforma. El contador de caracteres y el botón de envío con estado de carga muestran atención a detalles de usabilidad importantes para una herramienta de escritura libre.

EmotionInsights traduce el resultado técnico en una visualización sencilla. La emoción dominante se presenta como etiqueta principal y las emociones secundarias se muestran mediante barras. Esta representación es útil porque convierte números en información comprensible de forma inmediata. Por su parte, ConversationHistory permite navegar por mensajes anteriores, lo cual da continuidad temporal a la experiencia.

TopBar y StatusBanner completan la capa de interfaz. Uno expone identidad y salida de sesión; el otro comunica errores o mensajes informativos. Este tipo de componentes pequeños suele parecer secundario, pero en realidad son esenciales para que la aplicación se perciba ordenada y confiable.

31. Servicio de IA y contratos de intercambio

El servicio IA se organiza en torno a un contrato JSON sencillo. `/ia/analyze` responde a la inferencia básica y `/chat/message` agrega una capa más rica de contexto y recomendación. Desde el punto de vista del backend, esta estructura es ideal porque reduce ambigüedad en la comunicación y permite distinguir entre análisis puro y respuesta conversacional.

IAService traduce el texto, ejecuta el modelo y transforma la salida en un formato procesable. La clasificación emocional retorna probabilidades para varias emociones, lo cual permite ordenar y comparar resultados. El uso de un modelo preentrenado evita tener que entrenar desde cero un clasificador propio, reduciendo tiempo de desarrollo y haciendo posible una solución funcional con recursos limitados.

La recomendación textual generada por ChatService añade valor al análisis. No se entrega solamente un nombre de emoción, sino una orientación humana breve que contextualiza el resultado. Esa decisión convierte una salida técnica en una experiencia más cercana para el usuario final y refuerza la naturaleza de acompañamiento del sistema.

El contrato entre backend y IA debe mantenerse estable para garantizar la interoperabilidad. Si una de las partes modifica la forma del mensaje o la estructura del JSON, la otra deberá adaptarse. Por eso es importante tratar este servicio como una interfaz bien definida entre dos componentes independientes.

32. Escenarios de funcionamiento

Un primer escenario corresponde al usuario nuevo. El usuario abre la plataforma, crea una cuenta, inicia sesión y escribe su primer mensaje. En ese punto el sistema crea el perfil, genera un token, analiza el texto y guarda la primera interacción. Este flujo demuestra el recorrido completo de un caso de uso básico de extremo a extremo.

Un segundo escenario es el del usuario recurrente. En este caso, el usuario inicia sesión y consulta su historial antes de escribir un nuevo mensaje. El sistema recupera conversaciones previas, le permite revisar estados emocionales pasados y luego añade un nuevo análisis a la cronología. Esta funcionalidad aporta continuidad y hace que la aplicación tenga memoria útil.

Un tercer escenario cubre la falla del servicio de IA. Si FastAPI no responde o devuelve error, el backend no se detiene: usa el analizador local. El usuario sigue recibiendo una salida razonable, aunque menos sofisticada. Este comportamiento es importante porque evita una mala experiencia y protege el flujo principal de interacción.

Estos escenarios ayudan a explicar el valor real del sistema. La aplicación no solo ejecuta tareas técnicas; también responde de forma coherente a situaciones normales y a condiciones de excepción. Esa combinación de flujo principal y recuperación ante fallos es una señal de madurez arquitectónica.

33. Mantenibilidad, riesgos y limitaciones

Como todo sistema software, Emotional Chatbot tiene limitaciones. La principal es que el análisis emocional no sustituye una valoración profesional. Otra limitación es que la exactitud depende del modelo externo o del conjunto de palabras clave, lo cual puede llevar a resultados aproximados y no definitivos. Estas fronteras deben expresarse claramente en la documentación y en la interfaz.

En términos de mantenimiento, la principal recomendación es conservar el contrato de datos entre capas. También conviene centralizar configuraciones de URL, puertos y credenciales en archivos de entorno. Esto reduce errores y facilita despliegues distintos. La documentación actual ya deja ver una arquitectura propicia para este tipo de mantenimiento ordenado.

Existe además un riesgo operativo asociado a la dependencia de tecnologías externas como el modelo de Hugging Face o la traducción automática. Para reducirlo, el sistema ya integra un fallback local, pero podrían añadirse colas, cachés o reintentos controlados. Estas mejoras serían útiles si el proyecto se usa en un contexto más exigente que el académico.

A pesar de esas limitaciones, la solución ofrece una base funcional y pedagógica muy valiosa. Su claridad conceptual y su separación modular hacen que sea apropiada tanto para explicar principios de ingeniería de software como para demostrar integración entre frontend, backend e inteligencia artificial.

34. Glosario técnico breve

JWT: mecanismo de autenticación basado en tokens firmados que permite validar al usuario sin mantener sesiones en memoria del servidor.

DTO: objeto de transferencia de datos usado para comunicar capas sin exponer directamente entidades internas.

Fallback: mecanismo alternativo que se activa cuando el componente principal falla o no responde.

Repositorio: abstracción de acceso a datos que encapsula operaciones de lectura y escritura.

Stateless: modelo de interacción en el que el servidor no conserva el estado de sesión entre peticiones.

Transformers: biblioteca de aprendizaje automático usada para modelos de lenguaje y clasificación.

CORS: política de seguridad del navegador que controla qué orígenes pueden acceder a un recurso remoto.

FastAPI: framework Python para construir APIs de alto rendimiento.

Spring Security: módulo de Spring para autenticación y autorización.

Tailwind CSS: framework de utilidades para diseño de interfaces.

Astro: framework frontend orientado a rendimiento y composición de componentes.

React: biblioteca para construir interfaces de usuario basadas en componentes.

35. Desglose narrativo del flujo de autenticación

El flujo de autenticación puede describirse como una secuencia cuidadosamente encadenada entre la interfaz, la lógica de negocio y la persistencia. El usuario ingresa sus datos en AuthPanel; el frontend envía la solicitud al backend; AuthService normaliza el correo, verifica si la cuenta existe y decide si procede el registro o el inicio de sesión. Esta secuencia mantiene la responsabilidad distribuida y evita mezclar validación visual con reglas de negocio.

En el registro, la creación del usuario no solo escribe datos en la base, sino que establece la primera capa de seguridad del sistema. Al cifrar la contraseña, el backend protege la información sensible incluso si la base de datos llegara a exponerse. En el inicio de sesión, el sistema valida credenciales y genera un token que se convierte en la llave de acceso a la experiencia protegida. Desde ese momento, el usuario puede consultar su historial y analizar nuevos mensajes sin repetir credenciales en cada interacción.

Este flujo también representa una buena práctica desde la experiencia de usuario. Un formulario simple, una respuesta clara y una transición a la pantalla principal generan una interacción intuitiva. Para la documentación técnica, este escenario sirve como ejemplo de separación de responsabilidades, manejo de datos sensibles y diseño orientado al estado de autenticación.

36. Desglose narrativo del flujo de análisis emocional

El análisis emocional comienza cuando el usuario escribe un mensaje en ConversationWorkspace y decide enviarlo. La interfaz valida que exista texto no vacío y luego llama a la función de análisis centralizada en el store. A partir de allí, la solicitud pasa al backend, donde ConversationController verifica autenticación y delega la operación a ConversationService.

Dentro del servicio, el sistema intenta primero comunicarse con el microservicio de IA. Si la respuesta llega correctamente, se extraen la emoción dominante, la lista de emociones y la recomendación textual. El backend transforma esos datos en un DTO de salida, los serializa para persistencia y almacena la conversación. Finalmente, la interfaz recibe la respuesta, actualiza el panel de conversación y refresca la visualización de insights.

Si el servicio remoto falla, la experiencia sigue funcionando gracias al análisis local. Este detalle es particularmente relevante para una documentación extensa, porque demuestra que el sistema no está concebido como una suma frágil de partes, sino como una solución que intenta mantener su propósito incluso en condiciones adversas. El usuario sigue recibiendo una orientación aproximada y el sistema conserva el historial.

37. Descripción ampliada del historial de conversaciones

ConversationHistory no es solo una lista visual; es una puerta de acceso a la memoria funcional del sistema. Cada elemento representa una conversación que el usuario realizó en el pasado, y al seleccionar una de ellas la pantalla principal puede rehidratar la información para mostrar el mensaje y la respuesta asociados. De esta forma, el sistema no obliga al usuario a recordar todo manualmente.

La ordenación cronológica facilita la lectura de evolución emocional. Ver las conversaciones desde la más antigua hasta la más reciente permite identificar patrones, repeticiones o cambios de tono. En un sistema de acompañamiento emocional, esa continuidad temporal resulta especialmente útil, porque convierte la aplicación en un repositorio dinámico de interacciones personales.

Desde la perspectiva de implementación, el historial también ayuda a validar el sistema durante pruebas. Permite revisar qué se almacenó, cómo se procesó y si la interfaz reconstruye correctamente la información desde la base de datos. Esa capacidad de inspección es valiosa tanto para el desarrollo como para la sustentación del proyecto.

38. Consideraciones éticas y de uso responsable

El proyecto incluye mensajes explícitos indicando que no se trata de una herramienta clínica. Esta advertencia es necesaria porque el análisis de emociones puede ser interpretado erróneamente como un diagnóstico, y no lo es. El sistema ofrece orientación general, pero no reemplaza atención psicológica, psiquiátrica ni médica. Esta precisión debe aparecer en la documentación porque forma parte de la responsabilidad del diseño.

Otra consideración ética importante es la manera en que se presentan las recomendaciones. Las respuestas del sistema deben ser prudentes, breves y no intrusivas. El objetivo no es persuadir al usuario, sino acompañarlo con una primera aproximación segura. Esa distinción entre apoyo emocional e intervención profesional define el alcance real del producto y protege al usuario de interpretaciones excesivas.

También resulta relevante el manejo de datos personales. El sistema almacena correo, nombre y conversaciones, por lo que una implementación real debería considerar políticas de privacidad, acceso restringido, protección de credenciales y eventualmente anonimización. Aunque el proyecto tiene un alcance académico, estas reflexiones son indispensables en cualquier documentación técnica seria.

39. Pruebas sugeridas y validación funcional

Una documentación técnica robusta no solo debe describir lo que el sistema hace, sino también cómo puede validarse. En este caso se recomiendan pruebas de autenticación, pruebas de validación de formularios, pruebas de análisis emocional y pruebas de historia de conversaciones. Cada prueba debería comprobar que el backend responde correctamente y que el frontend refleja el estado esperado.

Para la autenticación, conviene verificar que un registro exitoso cree el usuario, que un correo duplicado sea rechazado y que credenciales inválidas no generen token. Para el módulo de conversación, se recomienda probar mensajes cortos, largos, vacíos y con múltiples emociones. También es importante ensayar el comportamiento cuando el servicio de IA está caído para confirmar que el fallback local responde adecuadamente.

En el frontend, las pruebas deben validar la alternancia entre login y registro, el comportamiento del loading, la visualización del historial y la correcta selección de conversaciones. Estas verificaciones no requieren una infraestructura compleja, pero sí ayudan a demostrar la calidad del sistema y la consistencia del diseño implementado.

40. Recomendaciones para despliegue y operación

Un despliegue limpio debería conservar separados los tres componentes principales. El frontend puede alojarse en un entorno de aplicaciones web estáticas o modernas; el backend en una instancia Java con acceso a base de datos; y el servicio FastAPI en un proceso independiente. Esta separación simplifica escalamiento, monitoreo y mantenimiento.

También es recomendable documentar las variables de entorno críticas, como URL base de la IA, credenciales de base de datos, puertos de operación y orígenes permitidos por CORS. Cuando estos valores quedan explícitos, el despliegue deja de depender de supuestos y se vuelve reproducible. Eso es esencial en entornos académicos donde los evaluadores pueden intentar ejecutar el proyecto en otro equipo.

Durante la operación cotidiana conviene revisar logs del backend, disponibilidad del servicio IA y tiempos de respuesta del análisis. Si el sistema se amplía con más usuarios, sería adecuado introducir monitoreo básico y rotación de registros. Estas prácticas no solo mejoran el mantenimiento, sino que también ayudan a justificar la calidad técnica del proyecto ante terceros.

41. Síntesis final extendida

La revisión del proyecto permite concluir que Emotional Chatbot no es un simple formulario con respuesta automatizada. Es una solución que articula autenticación, persistencia, visualización y análisis textual en una arquitectura separada por responsabilidades. Esa composición da como resultado un sistema funcionalmente claro y técnicamente coherente.

El backend gestiona el núcleo de la lógica. El frontend ordena la interacción. La IA agrega una capa de interpretación emocional. El mecanismo de respaldo protege la continuidad del servicio. La combinación de estas piezas refleja una intención de construir una plataforma completa y defendible desde la ingeniería de software.

Con la documentación ampliada, el proyecto queda mejor preparado para ser presentado como entrega universitaria o como base de desarrollo futuro. La estructura general puede seguir creciendo sin perder orden, precisamente porque desde su diseño se reconocen las capas, los contratos y las responsabilidades de cada componente.